

Experimento 2 — Reaproveitamento do INMET

Persistência localizada na Gold fato_internacao_meteorologia

10 de setembro de 2026

Sumário

1	Resultado e escopo	1
2	Baselines e comparação esperada	1
3	Comportamento encontrado antes da alteração	2
4	Reutilizações identificadas	2
5	DataFrame escolhido e estratégia	3
6	Materialização e liberação	3
7	Instrumentação adicionada	3
8	Testes criados e equivalência	4
9	Resultados técnicos	4
10	Arquivos e resumo do diff	5
11	Riscos e limites da interpretação	5
12	Comandos recomendados	5
12.1	Reproduzir os testes sintéticos	5
12.2	Executar somente a Gold meteorológica real posteriormente	5
12.3	Executar o DQ depois da Gold, quando autorizado	6
13	Situação final	6

1 Resultado e escopo

Foi implementada persistência exclusivamente do **df_inmet preparado** em `src/gold/fato_internacao_meteorologia.py`. A decisão foi precedida pela leitura integral do script e confirmação de reutilização da mesma linhagem.

A alteração de produção contém **43 linhas adicionadas**, sem remoção ou alteração das expressões funcionais existentes. O Experimento 1 e a DAG permaneceram intactos. Os testes sintéticos confirmaram equivalência dos resultados e uso efetivo do cache no plano Spark.

Não foram executados a DAG completa ou a Gold meteorológica real. Não houve leitura dos 8,4 milhões de registros, sobrescrita de datasets, alterações de permissões, commit, push ou reset. O ganho de desempenho ainda será medido pelo usuário.

2 Baselines e comparação esperada

O Experimento 1 permanece com o resultado observado de 273,001s para 57,380s no bloco de validação da fato de internação. Sua implementação não foi modificada por este experimento.

Para a Gold meteorológica, há duas observações anteriores: **525,490s** na run de 10/09/2026 01:37 UTC e **751,996s** na run de 10/09/2026 23:06 UTC. A segunda é a mais recente antes do Experimento 2. A variação entre elas reforça a necessidade de não atribuir causalidade a uma única comparação.

Métrica	Antes	Depois
Persistência df_inmet	Não	Sim
Persistência df_fato	Não	Não
Regras de validação	Iguais	Iguais
Join	Igual	Igual
Granularidade esperada	8.409.047	8.409.047
Tempo validação de chave INMET	Aproximadamente 79,828s	A medir
Tempo Gold meteorologia	525,490s / 751,996s	A medir

Os 8.409.047 registros são a cardinalidade esperada do dataset real. Os testes verificaram preservação da cardinalidade em dados sintéticos, não executaram essa quantidade de registros.

O intervalo de 79,828s é um baseline observacional da validação de chave e duplicidades. O novo cronômetro de validações também inclui período e correspondência de estações; os dois intervalos **não são diretamente equivalentes**. O log original de início da validação e a mensagem de duplicidades permanecem disponíveis para comparar o mesmo trecho.

3 Comportamento encontrado antes da alteração

O script faz uma única leitura do INMET diário, com mergeSchema=true e descoberta normal das partições, preservando a coluna data. Em seguida:

1. Confere as colunas obrigatórias.
2. Calcula total de registros e estações distintas sobre o INMET original.
3. Prepara duas chaves: _codigo_estacao_join com trim/upper/cast e _data_join com to_date.
4. Valida chaves, duplicidades e período.
5. Compara estações da fato e do INMET por dois anti-joins.
6. Deriva df_inmet_join por select, aliases e flag _meteo_match.
7. Executa left join meteorológico com broadcast existente.
8. Executa métricas, validações, amostra e gravação sobre df_integrada.
9. Relê a Gold gravada em df_validacao.

Não havia cache ou persistência explícita. df_inmet_join e df_integrada são DataFrames derivados: suas actions podem voltar a consumir a linhaagem do df_inmet preparado. Não existe nova leitura do INMET que contorne essa linhaagem.

4 Reutilizações identificadas

No caminho normal de sucesso, após a preparação, foram identificadas **nove actions consumidoras da linhaagem**:

Uso	Action
Chaves INMET inválidas	filter(...).count()
Duplicidades das chaves	groupBy(...).count().filter(...).count(); a última contagem é a action
Período mínimo/máximo	collect() da agregação
Estações da fato sem INMET	count() do anti-join
Estações INMET sem uso na fato	count() do anti-join
Estatísticas do join meteorológico	collect() da agregação
Dias meteorológicos incompletos	count() sobre df_integrada
Amostra da Gold integrada	show()
Gravação da Gold integrada	write.parquet()

São cinco actions de validação/correspondência e quatro no fluxo derivado do join. Há ainda actions diagnósticas condicionais de show() nos caminhos de erro.

As duas contagens iniciais, `total_inmet` e `total_estacoes_inmet`, continuam antes da persistência e não são beneficiadas por ela. Não foram movidas para preservar a ordem existente. Não foi adicionado um count extra para materializar o cache.

5 DataFrame escolhido e estratégia

O alvo é `df_inmet` após as duas transformações da seção 4, incluindo as colunas originais e as chaves normalizadas. A persistência é:

```
inicio_reutilizacao_inmet = perf_counter()
df_inmet = df_inmet.persist(StorageLevel.MEMORY_AND_DISK)
```

Foi escolhido `StorageLevel.MEMORY_AND_DISK`, explicitamente, porque:

- O INMET diário tem cardinalidade esperada de 43.840 linhas, muito inferior às 8.409.047 interações da fato.
- Existem reutilizações concretas até a escrita.
- O ambiente executa em local[2], compartilhando recursos locais.
- A possibilidade de armazenamento em disco evita depender exclusivamente da disponibilidade de memória.

Não foi medida a ocupação real em bytes do DataFrame. O ganho não é garantido: materialização, serialização, leitura de cache e eventual uso de disco também têm custo.

Não foi persistido `df_fato`, `df_integrada` ou `df_inmet_join`. Apenas `df_inmet` recebe uma chamada de `persist`.

6 Materialização e liberação

`Persist` é lazy. A contagem já existente de chaves inválidas, `chaves_inmet_invalidas`, é a primeira action após a persistência e materializa o INMET preparado. A contagem original foi mantida sem alteração.

O cache permanece ativo durante as validações e todas as actions dependentes do join. Sua liberação ocorre depois da gravação da Gold e antes da validação pós-gravação:

```
fim_reutilizacao_inmet = perf_counter()
df_inmet.unpersist()
```

A escrita é a última action do caminho de sucesso que usa a linhagem do INMET. As validações seguintes usam `df_validacao`, obtido da leitura da Gold já gravada; liberar o cache nesse ponto não antecipa sua última utilização.

Não foi introduzido try/finally abrangendo mais de mil linhas do script. Em caso de exceção antes da escrita, o `unpersist` explícito não é alcançado; no uso standalone por `spark-submit`, o encerramento da aplicação libera os recursos do Spark. Se o script fosse incorporado a um processo com `SparkSession` duradoura e exceções capturadas externamente, seria necessário rever esse ciclo de vida. Os testes garantem sua própria limpeza em finally, inclusive nos erros esperados.

7 Instrumentação adicionada

Foram usados apenas `perf_counter` da biblioteca padrão e quatro logs:

Log	Intervalo efetivamente medido
INMET preparação/materialização	Início da preparação até a primeira contagem de chaves inválidas concluída
INMET validações (chave, período e estações)	Início da seção 5 até a aprovação da comparação de estações
Join meteorológico (até agregação de validação)	Construção do join até <code>collect()</code> das estatísticas existentes
INMET reutilização total (persistência até escrita)	Imediatamente antes de <code>persist</code> até a conclusão da escrita

A materialização inclui preparação, registro do cache e primeira action; não inclui a leitura inicial e as duas contagens anteriores à preparação.

O cronômetro do join mede execução efetiva e agregação de validação, não somente a construção lazy do plano. Não representa tempo exclusivo do algoritmo de join nem todos os usos posteriores do join.

Os intervalos **se sobrepõem e não devem ser somados**. O tempo total de reutilização inclui trabalho na fato, validações, logs, amostra e escrita. O instante final é capturado antes de unpersist, portanto exclui o tempo de liberação.

8 Testes criados e equivalência

Arquivo: `tests/test_fato_meteorologia_persistencia.py`, com 165 linhas.

O teste compila somente os imports, o helper de validação de colunas, a preparação da fato e o fluxo real desde as métricas iniciais do INMET até antes da escrita. Não importa o pipeline inteiro e não executa suas leituras ou escrita.

A variante sem persistência usa o mesmo fluxo de produção, retirando apenas a chamada persist na AST. A variante com persistência executa a implementação real. No teste, `collect()` sobre cinco interações sintéticas representa a última action de consumo; em seguida é executado o trecho real de liberação.

Foram criados três testes:

1. **Equivalência de métricas, join, flags e granularidade.** Compara schema, linhas, logs funcionais, contagem INMET, estações distintas, chaves, duplicidades, período, anti-joins, estatísticas do join e dias incompletos.
2. **Reprovações preservadas**, com cinco subcasos: chave nula, duplicidades, período incorreto, ausência de correspondência dentro do período e quantidade de observações diferente de 24.
3. **Escopo e ordem da liberação.** Confirma uma persistência somente em `df_inmet`, um unpersist depois da escrita e ausência de consumidores dessa linhagem depois da liberação.

O caso válido usa três dias INMET e cinco interações: três com match, uma anterior a 2023 e uma posterior a 2025. Duas interações compartilham a mesma chave meteorológica e permanecem como duas linhas. Há uma temperatura nula em dia com match para verificar que ausência de medição não remove a interação.

Além da igualdade dos resultados, o teste verificou `StorageLevel.MEMORY_AND_DISK`, ausência de persistência da fato, presença de `InMemoryTableScan no plano executado` e liberação do cache após a última action sintética.

9 Resultados técnicos

Verificação	Resultado
Experimento 2	3 testes, incluindo 5 subcasos de erro: OK, 21,087s
Experimento 1	6 testes: OK, 13,529s
Orquestração e DAG	5 testes: OK, 3,721s
Compilação do script e novo teste	OK
Imports isolados	OK
airflow dags list-import-errors	No data found
TaskGroups / tasks	5 / 24
git diff -check	OK

Os testes Spark usaram spark-submit, Spark 3.5.0 e Java 17. Foram emitidos avisos de sockets e biblioteca Hadoop nativa, sem falhas nos testes.

Uma comparação da AST com HEAD confirmou que, removidas somente as adições de persistência, liberação e instrumentação, **o código funcional restante é idêntico**. Isso cobre joins, broadcast, shuffle, repartition, paths, escrita, condições e regras meteorológicas.

A primeira tentativa dessa comparação excedeu o limite de tamanho de argumentos do Windows. O baseline foi então transmitido pela entrada padrão; a comparação foi concluída com sucesso, sem alteração do ambiente.

Hashes SHA-256 preservados:

DAG:
7BEE4CF50BBE3422F1B0030230D5B1266AC39AF926DF65D8F16ACE573FC25738

Gold fato_internacao (Experimento 1):
F188EC6AB7F6AE60A9A34FBA60A497F777C016915B5ED56C6A4967B4029D4FA8

10 Arquivos e resumo do diff

Modificado:

- src/gold/fato_internacao_meteorologia.py: **43 inserções**, somente imports, persistência, liberação e instrumentação.

Criados:

- tests/test_fato_meteorologia_persistencia.py: **165 linhas**.
- docs/Relatorio_Experimento_2_Persistencia_INMET.md.
- docs/Relatorio_Experimento_2_Persistencia_INMET.pdf.

O git diff --stat dos arquivos rastreados mostrou:

```
src/gold/fato_internacao_meteorologia.py | 43 ++++++
1 file changed, 43 insertions(+)
```

Os arquivos novos não rastreados não aparecem nesse comando. O diff integral de produção e o teste foram revisados. Nenhuma alteração foi feita na DAG ou no Experimento 1.

11 Riscos e limites da interpretação

- O cache pode reduzir releituras e recomputações, mas não elimina o custo de varrer a fato, dos joins, de amostras ou da escrita.
- Não se promete eliminar os 79,828s do intervalo histórico.
- As duas contagens anteriores à preparação continuam fora do cache.
- MEMORY_AND_DISK pode consumir disco se não houver memória suficiente; local[2] não isola CPU, memória ou I/O das demais aplicações.
- O uso explícito de unpersist está no caminho de sucesso; o encerramento da aplicação é responsável pela liberação em falhas anteriores.
- O experimento pressupõe entradas estáveis durante a aplicação. Alterações concorrentes no INMET não foram avaliadas.
- A cardinalidade real, a aprovação do DQ e o tempo real continuam pendentes de execução pelo usuário.
- A redução esperada de recomputação é sustentada pela reutilização e pelo plano sintético; ainda não há ganho de performance medido para este experimento.

12 Comandos recomendados

12.1 Reproduzir os testes sintéticos

```
docker compose -f docker-compose.airflow.yaml exec -T -w /home/jovyan/work airflow-worker
↳ spark-submit --master "local[2]" --conf spark.ui.enabled=false
↳ tests/test_fato_meteorologia_persistencia.py -v
```

12.2 Executar somente a Gold meteorológica real posteriormente

Este comando não foi executado. Ele sobrescreve a saída Gold meteorológica conforme a escrita existente. Preserva as configurações da task na DAG; não executa automaticamente seu DQ.

```
docker compose -f docker-compose.airflow.yaml exec -T airflow-worker spark-submit --master
↳ "local[2]" --conf spark.ui.enabled=false --conf spark.sql.shuffle.partitions=40 --name
↳ tcc-gold-fato-internacao-meteorologia --deploy-mode client
↳ /home/jovyan/work/src/gold/fato_internacao_meteorologia.py
```

12.3 Executar o DQ depois da Gold, quando autorizado

```
docker compose -f docker-compose.airflow.yaml exec -T airflow-worker spark-submit --master  
↪ "local[2]" --conf spark.ui.enabled=false --conf spark.sql.shuffle.partitions=4 --name  
↪ tcc-dq-gold-fato-internacao-meteorologia --deploy-mode client  
↪ /home/jovyan/work/src/quality/gold/dq_fato_internacao_meteorologia.py
```

Para medir a duração da task diretamente comparável aos metadados anteriores, utilizar a execução autorizada pelo Airflow e consultar seus logs. Uma execução direta por spark-submit tem custos externos diferentes.

Registrar os quatro logs de performance, a duração da task e a aprovação do DQ. Comparar o mesmo intervalo de validação com o baseline de 79,828s. Repetições com entradas e condições comparáveis ajudam a distinguir ganho da alteração de variações da máquina.

13 Situação final

Experimento 2 implementado, com equivalência sintética e escopo estático aprovados. Nenhuma execução sobre os dados reais foi realizada. O usuário fará a execução real após revisar este relatório.